ASSIGNMENT 5

NAME: JANANIPRIYA G
REG.NO: 192324041
SUBMISSION DATE:16/11/26
SLOT: A
SUBJECT:JAVA PROGRAMMING
COURSE CODE: CSA0903

PART A – PERSISTENCE DESIGN & PSEUDOCODE

TASK A1 – DESIGN A FILE FORMAT

Transport Office Bus-Pass Records (7 Marks)

A1.1 – Represent the Record (2 Marks)

Each bus-pass record contains seven fields: pass ID, student register number, name, route number, boarding stop, validity date and amount paid.

(a) Delimited Text File Format

The records can be stored in a CSV-style text file using the pipe (|) symbol as the delimiter.

Field Order:

1. Pass ID
2. Student Register Number
3. Student Name
4. Route Number
5. Boarding Stop
6. Validity Date
7. Amount Paid

Example Record:

P1001|192311001|Arun Kumar|12|Tambaram|2027-03-31|4500.00

P1002|192311002|Priya Devi|7|Chromepet|2027-03-31|3800.00

Handling Names Containing the Delimiter:

If a name or another text field contains the | symbol, it must be escaped using a backslash (\). Backslashes must also be escaped. During reading, the program reverses this escaping process.

For example:

Original Name: Arun|Kumar

Stored Name: Arun\|Kumar

This prevents the delimiter inside a field from being treated as a field separator.

(b) Binary File Format Using DataOutputStream

The binary file stores the fields in the following order:

Field	Java Data Type	Storage Format
Pass ID	String	writeUTF()
Register Number	String	writeUTF()
Student Name	String	writeUTF()
Route Number	int	writeInt() – 4 bytes
Boarding Stop	String	writeUTF()
Validity Date	String	writeUTF()
Amount Paid	double	writeDouble() – 8 bytes

The writeUTF() method stores a length prefix followed by the encoded string data. It does not use a fixed field size; its length prefix is two bytes.

Advantages of Binary Format:

- Stores numerical values in their binary representation.
- Avoids repeatedly converting numbers into text.
- Can be more compact for suitable data.
- Requires matching DataInputStream methods to read the records.

A1.2 – Design the Operations (3 Marks)

The text-file design uses BufferedReader and BufferedWriter to read and write records efficiently.

1. Pseudocode for Adding a New Pass

Algorithm: AddPass

Input: Pass ID, register number, name, route number, boarding stop, validity date and amount.

Output: New pass record stored in the file.

BEGIN

INPUT passID

INPUT registerNo

INPUT name

INPUT routeNo

INPUT boardingStop

INPUT validityDate

INPUT amount

VALIDATE all input fields

IF any field is invalid THEN

DISPLAY "Invalid pass details"

STOP

END IF

ESCAPE delimiter characters in text fields

OPEN pass file using BufferedWriter

in append mode

WRITE all seven fields separated by "|"

WRITE newline

CLOSE the file

DISPLAY "Pass added successfully"

END

Stream Classes Used:

- FileWriter – writes character data into the text file.
- BufferedWriter – improves writing efficiency by buffering characters.
- Append mode – adds new records without deleting existing records.

2. Pseudocode for Finding a Pass by ID

Algorithm: FindPass

Input: Pass ID.

Output: Matching pass record or not-found message.

BEGIN

INPUT searchPassID

OPEN pass file using BufferedReader

WHILE another line is available DO

READ one line

SPLIT the line using the delimiter

while respecting escaped delimiters

IF the number of fields is not equal to 7 THEN

SKIP the damaged line

CONTINUE

END IF

DECODE escaped text fields

IF first field equals searchPassID THEN

DISPLAY the complete pass record

CLOSE the file

STOP

END IF

END WHILE

CLOSE the file

DISPLAY "Pass not found"

END

Stream Classes Used:

- FileReader – reads character data from the text file.
- BufferedReader – reads records line by line efficiently.
- Line-by-line reading avoids loading the entire file into memory.

3. Pseudocode for Listing Passes by Route

Algorithm: ListPassesByRoute

Input: Route number.

Output: All pass records belonging to the selected route.

BEGIN

```
INPUT searchRoute

SET found = FALSE

OPEN pass file using BufferedReader

WHILE another line is available DO

  READ one line

  SPLIT the line into fields

  IF the number of fields is not equal to 7 THEN
    SKIP the damaged line
    CONTINUE
  END IF

  IF route number equals searchRoute THEN

    DISPLAY the complete pass record
    SET found = TRUE

  END IF

END WHILE

CLOSE the file
```

```
  IF found = FALSE THEN
    DISPLAY "No passes found for this route"
  END IF


END
```

Stream Classes Used:

- BufferedReader – reads the records line by line.
- FileReader – connects the program to the character-based file.

Note: FileInputStream and FileOutputStream are byte-oriented streams. They are more suitable for binary data rather than ordinary character-based text records.

A1.3 – Design for Damaged Data (2 Marks)

The file system must handle damaged records, missing files and incomplete writes safely.

Problem	Detection	Handling
Damaged line	Check whether the line contains exactly seven valid fields.	Skip the damaged line and continue reading the remaining records.
Missing file	Check whether the file exists before reading.	Display a suitable error or create a new file when adding records.
Invalid numerical value	Check whether route number and amount can be converted into valid numerical values.	Reject the record and report the invalid data.
Incomplete write	Detect incomplete records using field validation and a record checksum or format validation.	Preserve the original file and recover from a verified backup or temporary file.

Handling an Incomplete Write

To prevent data loss during file updates:

1. Write updated records into a temporary file.

2. Flush and close the temporary file.
3. Validate the temporary file.
4. Replace the original file with the temporary file using an atomic move where supported.
5. Keep a backup of the previous valid file.

This reduces the risk of losing all records when power failure occurs during an update.

Alternative Concept and Complexity Analysis

Instead of using a delimited text file, the bus-pass records can be stored in a RandomAccessFile with fixed-length records.

RandomAccessFile Approach

Each record is assigned a fixed record size. For example:

Field	Allocated Size
Pass ID	10 bytes
Register Number	15 bytes
Name	40 bytes
Route Number	4 bytes
Boarding Stop	30 bytes
Validity Date	10 bytes
Amount	8 bytes
Total	117 bytes

Text fields are padded to their allocated lengths. If a field is longer than its allocated size, it must be rejected or handled using a defined encoding and overflow policy.

Finding a Record:

If the record index is known, the program calculates its position and uses seek().

```
RandomAccessFile file =
    new RandomAccessFile("passes.dat", "rw");


int recordSize = 117;
```

int recordIndex = 5;

file.seek((long) recordIndex * recordSize);

This directly accesses the required record position.

Comparison

Feature	Delimited Text File	RandomAccessFile
Record storage	Variable-length	Fixed-length
Reading	Sequential	Direct access possible
Finding by ID	O(n)	O(1) seek if record index is known
Space usage	Variable	Fixed record size
Human readability	Easy	Not directly readable
Updating records	May require rewriting file	Direct update possible
Field padding	Not required	Required

Time Complexity:

- Text-file search: O(n), because records are checked sequentially.
- RandomAccessFile search: O(1) to seek to a known record index.
- If only the pass ID is known and no index exists, RandomAccessFile still requires an O(n) scan.

Space Complexity:

- Text file: Depends on the actual size of the stored records.
- RandomAccessFile: O(n × recordSize), where unused padding may waste storage.

Conclusion: A delimited text file is suitable for simple bus-pass record storage and easy inspection. RandomAccessFile is more suitable when fixed-size records and direct access by known record position are required.

JAVA Program

import java.io.*;

```
public class LineCounter {
   public static void main(String[] args) {
      String fileName = "attendance.txt";
      int lineCount = 0;

      try (BufferedInputStream bis =
            new BufferedInputStream(
               new FileInputStream(fileName))) {

         byte[] buffer = new byte[8192];
         int bytesRead;

         while ((bytesRead = bis.read(buffer)) != -1) {
            for (int i = 0; i < bytesRead; i++) {
               if (buffer[i] == '\n') {
                  lineCount++;
               }
            }
         }

         System.out.println("Total lines: " + lineCount);

      } catch (IOException e) {
         System.out.println("File error: " + e.getMessage());
      }
   }
}
```

```
import java.sql.*;
import java.util.*;

public class StudentDao implements Dao<Student, Integer> {

  private Connection con;

  public StudentDao(Connection con) {
    this.con = con;
  }

  public void insert(Student s) throws SQLException {
    String sql = "INSERT INTO Student VALUES (?, ?, ?, ?)";

    try (PreparedStatement ps = con.prepareStatement(sql)) {
      ps.setInt(1, s.id);
      ps.setString(2, s.regNo);
      ps.setString(3, s.name);
      ps.setString(4, s.email);
      ps.executeUpdate();
    }
  }

  public Optional<Student> findById(Integer id)
      throws SQLException {

    String sql = "SELECT * FROM Student WHERE id = ?";
```

```
    try (PreparedStatement ps = con.prepareStatement(sql)) {
      ps.setInt(1, id);

      try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
          Student s = new Student(
            rs.getInt("id"),
            rs.getString("reg_no"),
            rs.getString("name"),
            rs.getString("email")
          );

          return Optional.of(s);
        }
      }
    }

    return Optional.empty();
}

public List<Student> findAll() throws SQLException {
    List<Student> students = new ArrayList<>();

    String sql = "SELECT * FROM Student";

    try (Statement st = con.createStatement();
        ResultSet rs = st.executeQuery(sql)) {
```

```
        while (rs.next()) {
            students.add(new Student(
                rs.getInt("id"),
                rs.getString("reg_no"),
                rs.getString("name"),
                rs.getString("email")
            ));
        }
    }

    return students;
}

public void update(Student s) throws SQLException {
    String sql = "UPDATE Student SET name=?, email=? WHERE id=?";

    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, s.name);
        ps.setString(2, s.email);
        ps.setInt(3, s.id);
        ps.executeUpdate();
    }
}

public void delete(Integer id) throws SQLException {
    String sql = "DELETE FROM Student WHERE id=?";
```

```
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }
}
```